

JavaScript Final Test

Total Marks: 75 | Time Allowed: 75 Minutes

Section A: Multiple Choice Questions (20 Marks)

Choose the correct answer for each question. Each question is worth 2 marks.

1. What is the output of the following code?

JavaScript

```
console.log(typeof null);
```

- a) `"null"`
- b) `"object"`
- c) `"undefined"`
- d) `"function"`

2. Which of the following correctly describes the difference between `==` and `===` ?

a) `==` checks for value and type equality; `===` only checks for value equality.

b) `==` is used for assignment; `===` is used for comparison.

c) `==` compares values after performing type coercion; `===` compares both value and type without coercion.

d) There is no difference; they function identically.

3. What will be the value of `result` after this code executes?

JavaScript

```
const obj = { a: 1, b: 2 };  
const { c = 3 } = obj;  
const result = c;
```

- a) `undefined`

- b) 1
- c) 3
- d) `ReferenceError`

4. Consider the code below. What is printed to the console?

JavaScript

```
const arr = [1, 2, 3];  
const newArr = [...arr, 4];  
console.log(arr.length);
```

- a) 3
- b) 4
- c) `[1, 2, 3, 4]`
- d) `TypeError`

5. Which array method returns a new array containing only the elements that pass a specific condition?

- a) `map()`
- b) `forEach()`
- c) `reduce()`
- d) `filter()`

6. What happens when you add a duplicate value to a JavaScript `Set` ?

- a) The set throws a `SyntaxError` .
- b) The duplicate value replaces the original value.
- c) The duplicate value is ignored, and the size remains the same.
- d) The set automatically converts into an array.

7. What is the output of this code snippet?

JavaScript

```
console.log(1 + "2" + 3);
```

- a) 6
- b) "123"
- c) "15"
- d) NaN

8. Which statement about `const` is true?

- a) Variables declared with `const` cannot have their internal properties mutated (e.g., adding a property to an object).
- b) Variables declared with `const` are block-scoped and cannot be reassigned.
- c) `const` variables are hoisted and initialized as `undefined`.
- d) `const` can only store primitive data types.

9. What does the following code return?

JavaScript

```
const numbers = [10, 20, 30];  
const found = numbers.find(num => num > 15);  
console.log(found);
```

- a) [20, 30]
- b) 20
- c) true
- d) 1

10. What is the value of `x` after the logical OR assignment operator runs?

JavaScript

```
let x = 0;  
x ||= 10;
```

- a) 0
- b) 10
- c) undefined
- d) false

Section B: Conditionals and Data Types (10 Marks)

Question 11 (4 Marks) Analyze the following code snippet. Identify the final value of `discount` and briefly explain **why** that specific block executed.

JavaScript

```
let membership = "Gold";
let totalSpent = 45;
let discount = 0;

if (membership === "Platinum" || totalSpent > 100) {
    discount = 20;
} else if (membership === "Gold" && totalSpent > 50) {
    discount = 15;
} else if (membership === "Gold" || totalSpent > 20) {
    discount = 10;
} else {
    discount = 5;
}
```

Question 12 (6 Marks) Rewrite the following complex `if/else` structure into a clean, readable `switch` statement.

JavaScript

```
let systemRole = "editor";
let accessLevel;

if (systemRole === "admin") {
    accessLevel = "Full Access";
} else if (systemRole === "editor" || systemRole === "author") {
    accessLevel = "Write Access";
} else if (systemRole === "viewer") {
    accessLevel = "Read Only";
} else {
```

```
    accessLevel = "No Access";  
  }
```

Section C: Arrays and Objects (15 Marks)

Question 13 (7 Marks) You are managing a library database. Look at the data object below and write the JavaScript code to perform the following operations:

JavaScript

```
const library = {  
  name: "Central Library",  
  location: "Downtown",  
  books: [  
    { title: "The Hobbit", available: true },  
    { title: "1984", available: false }  
  ]  
};
```

- **a)** Change the `available` status of "1984" to `true`. (2 Marks)
- **b)** Write a line of code to add a new book object: `{ title: "Brave New World", available: true }` to the end of the `books` array. (2 Marks)
- **c)** Use **object destructuring** to extract the `name` and `location` properties from the `library` object into two standalone variables. (3 Marks)

Question 14 (8 Marks) Given the array of user objects below:

JavaScript

```
const users = [  
  { id: 1, name: "Alice", role: "user" },  
  { id: 2, name: "Bob", role: "admin" },  
  { id: 3, name: "Charlie", role: "user" }  
];
```

- **a)** Use an appropriate array iteration method to find the **first** user whose role is `"admin"` and store it in a variable called `adminUser`. (4 Marks)
- **b)** Use an array iteration method to create a **new array** containing only the names of the users (`["Alice", "Bob", "Charlie"]`). Store it in a variable called

userNames . (4 Marks)



Section D: Sets, Spread Operator, and Iterations (15 Marks)

Question 15 (5 Marks) Explain the core differences between a JavaScript **Array** and a JavaScript **Set**. In your answer, explicitly mention how they handle duplicates and how elements are accessed.

Question 16 (5 Marks) You are given two arrays representing grocery lists.

JavaScript

```
const basicItems = ["milk", "bread", "eggs"];
const extraItems = ["bread", "cheese", "apples"];
```

Using the **spread operator** and a **Set**, write a single line of code (or short block) to combine these two arrays into a single array called `uniqueGroceries` that contains **no duplicate items**.

Question 17 (5 Marks) Use the `.reduce()` method on the following array to calculate the total sum of all numbers. You must show the callback function and the initial value.

JavaScript

```
const prices = [5.99, 10.00, 2.50, 1.50];
```

Section E: Practical Application (15 Marks)

Question 18 (15 Marks) Scenario: You are building a student data processor for a school.

Write a function named `processStudents` that accepts an **array of student objects** as an argument. Each student object looks like this:

JavaScript

```
{ name: "John", score: 78, passed: false }
```

Your function must perform the following operational pipeline using clean, modern JavaScript syntax:

1. **Filter** out any students who have a score of less than 50 (they failed automatically).
2. For the remaining students, update their `passed` property to `true`.
3. **Return** the newly updated array of passing students without mutating the original input array (hint: use shallow copying/spread operator where necessary).

Test Case Example to guide your logic:

JavaScript

```
const batch = [  
  { name: "Alex", score: 45, passed: false },  
  { name: "Blake", score: 85, passed: false },  
  { name: "Casey", score: 60, passed: false }  
];
```

```
// Calling processStudents(batch) should yield:  
// [  
//   { name: "Blake", score: 85, passed: true },  
//   { name: "Casey", score: 60, passed: true }  
// ]
```

End of Assessment.